

第五章 数组和广义表

本章所讨论的数组与高级语言中的数组区别：

- 1、高级语言中的数组是顺序结构；
- 2、本章的数组既可以是顺序的，也可以是链式结构，用户可根据需要选择。

5.1 数组和线性表的关系以及数组的运算

5.2 数组的顺序存储结构

5.3 矩阵的压缩存储

5.1 数组和线性表的关系以及数组的运算

任何数组A都可以看作一个线性表

$$A=(a_1, a_2, \dots, a_i, \dots a_n)$$

一维数组n时,

a_i 是数组中第i个数据元素, $0 \leq i \leq n-1$;

二维数组m×n时,

行优先: a_i 是数组中第i行所有元素, $0 \leq i \leq m-1$, 表中每一个元素是一个一维数组;

列优先: a_i 是数组中第i列所有元素, $0 \leq i \leq n-1$, 表中每一个元素是一个一维数组;

三维数组时,

表中每一个元素 a_i 是一个二维数组;

n维数组时,

表中每一个元素 a_i 是一个(n-1)维数组

$$A_{m \times n} = \begin{pmatrix} \overbrace{(a_{00} \quad a_{01} \quad \dots \quad a_{0,n-1})} & \\ \overbrace{(a_{10} \quad a_{11} \quad \dots \quad a_{1,n-1})} & \\ \overbrace{(\dots \quad \dots \quad \dots \quad \dots)} & \\ \overbrace{(a_{m-1,0} \quad a_{m-1,1} \quad \dots \quad a_{m-1,n-1})} & \end{pmatrix}$$

数组与线性表之间的关系

线性表的扩展，其数据元素本身也是线性表

数组的特点

- 数组中各元素都具有统一的类型
- 可以认为， d 维数组的非边界元素具有 d 个直接前趋和 d 个直接后继
 - 以二维数组为例：上和左，下和右；多维以此类推
- 数组维数确定后，数据元素个数和元素之间的关系不再发生改变，适合于顺序存储
 - 什么情况下考虑链式存储或其他存储？
- 每组有定义的下标都存在一个与其相对应的值

在数组上的基本操作

- 给定一组下标，取得相应的数据元素值， Value
- 给定一组下标，修改相应的数据元素值， Assign

数组的基本操作定义

(1)构造n维数组 `InitArray(&A, dim, bound1, ..., boundn )`

(2)销毁数组A `DestroyArray(&A)`

(3)取得指定下标的数组元素值

`Value(A, &e, index1, ..., indexn )`

(4)为指定下标的数组元素重新赋值

`Assign(&A, e, index1, ..., indexn )`

使用变长参数来实现对多维数组的通用操作函数。

`InitArray`函数参数`dim`来指定数组的维度，同时确定参数数量

`DestroyArray`、`Value`、`Assign`三个函数，用数组`A`变量中的成员变量`dim`来保存数组的维度，同时确定参数的数量。

```
#include <stdarg.h>

typedef struct{
    ElemType * base;
    int dim;
    int * bounds;
    int * constants;
}Array;
```

1) 变长参数: 通过三个宏 (va_start、va_end、va_arg) 和一个类型 (va_list) 实现的。使用时需包含头文件<stdarg.h>

```
typedef char* va_list;  
void va_start ( va_list ap, prev_param ); /* ANSI version */  
type va_arg ( va_list ap, type );  
void va_end ( va_list ap );
```

2) 使用方法:

用 va_start 获取参数列表 (的地址) 存储到 ap 中,
用 va_arg 逐个获取值, 最后用 va_end 将 ap 置空。

```
__cdecl int va_sum (int first_num, ...){  
    va_list ap;                //(1) 定义参数列表ap  
    va_start(ap, first_num); //(2) ap指向参数列表  
    int nParaSum = first_num;  
    int temp = 0;  
    //(3)循环依次获取后续参数。问题: 如何确定参数数量?  
    while ((temp = va_arg(ap, int)) != -1)  
    {    //例子程序的最后一个参数是特殊值 -1, 作为结束符使用  
        //想想看, scanf和printf等如何确定参数个数?
```

```
1  /* file path: Microsoft Visual Studio 9.0\VC\include\vadefs.h */  
2  #ifdef __cplusplus  
3  #define _ADDRESSOF(v) ( &reinterpret_cast<const char &>(v) )  
4  #else  
5  #define _ADDRESSOF(v) ( &(v) )  
6  #endif  
7  
8  #define _INTSIZEOF(n) ( (sizeof(n) + sizeof(int) - 1) & ~(sizeof(int) - 1) )  
9  
10 #define _crt_va_start(ap,v) ( ap = (va_list)_ADDRESSOF(v) + _INTSIZEOF(v) )  
11 #define _crt_va_arg(ap,t) ( *(t *)((ap += _INTSIZEOF(t)) - _INTSIZEOF(t)) )  
12 #define _crt_va_end(ap) ( ap = (va_list)0 )
```

1) __cdecl, 强制使用c的参数入栈方式, 最右的先入栈; 系统栈是从高地址向低地址方向入栈的。

2) 三个宏和一个类型是如何做到可变参数的?

函数的所有参数均被从右向左压入栈中, 将ap指向第一个固定参数在栈中的位置后, 通过对ap指针值按参数类型的加, 来依次获得每个参数的指针; 再对指针指向的内存区域进行强制类型转换后得到所需参数值。

教材93-94的InitArray等函数实现都用了。

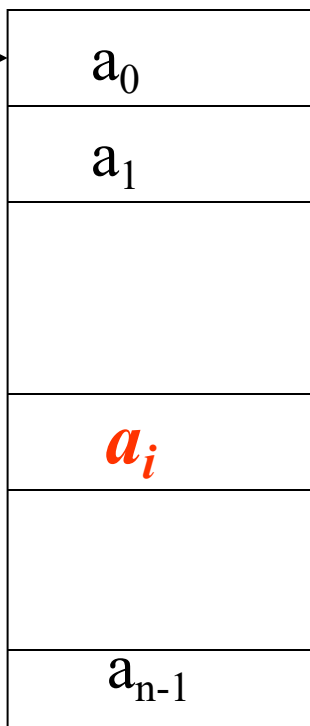
5.2 数组的顺序存储结构

一维数组

ElemType a[n];

基地址

b →



} L个存储单元大小

方法一： $LOC[i] = LOC[0] + i \times L$

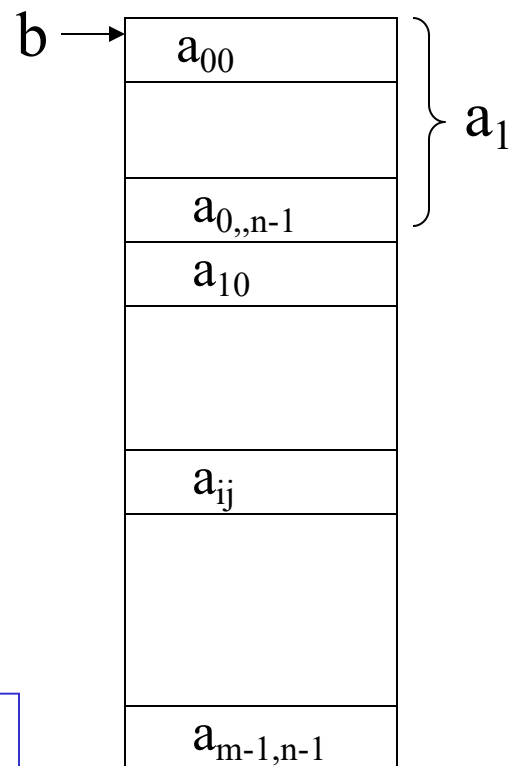
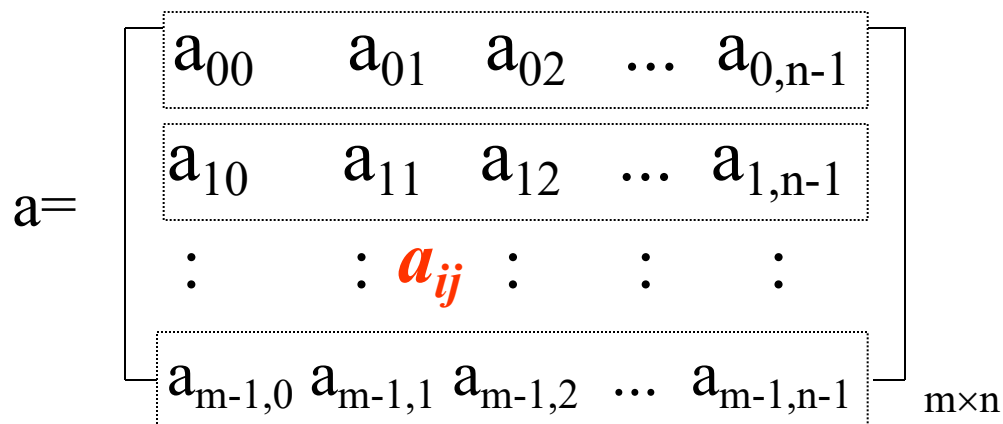
通式（方法二）：

$$LOC[i] = b + i \times L = c_0 + c_1 \times i$$

$$\begin{cases} c_1 = L & // \text{一维数组单个数据元素大小} \\ c_0 = b = LOC[0] & // \text{基地址} \end{cases}$$

二维数组

ElemType a[m][n];

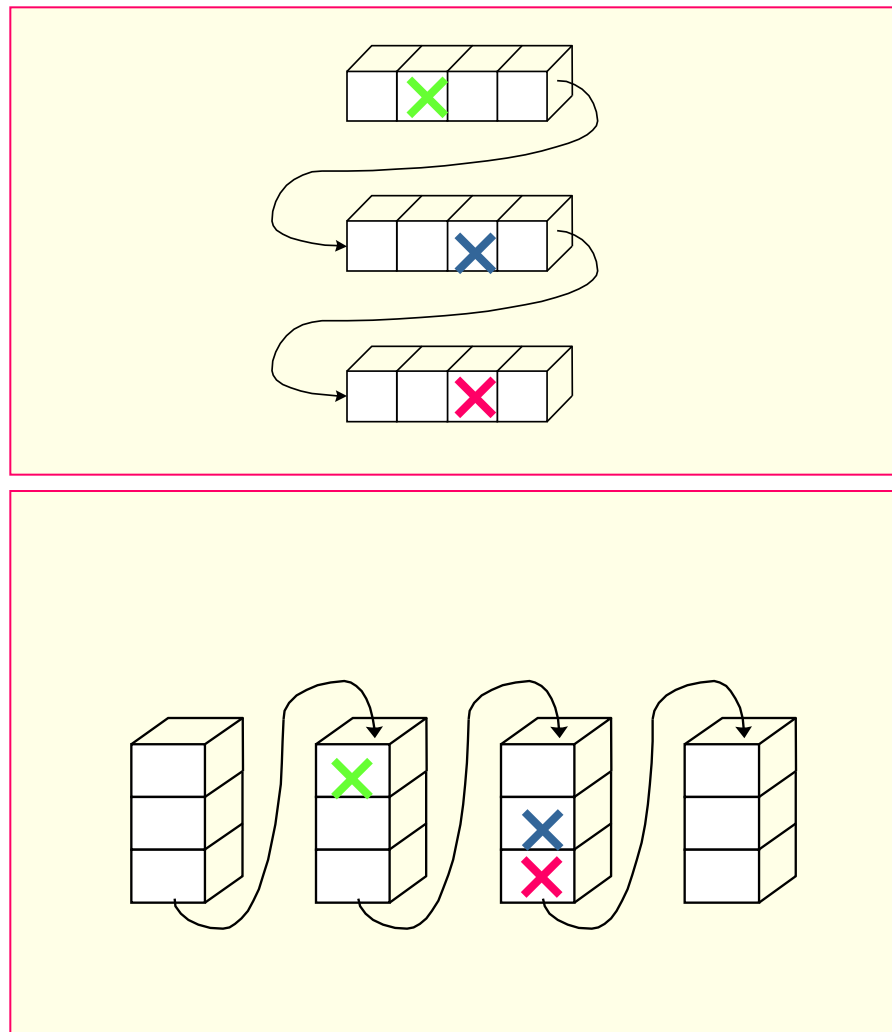
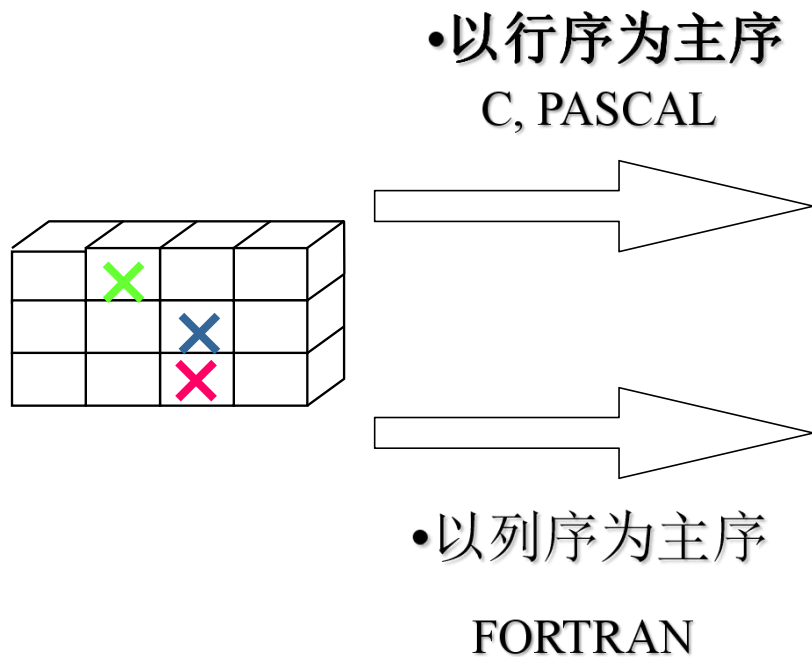


注：Pascal、C语言以行序为主序

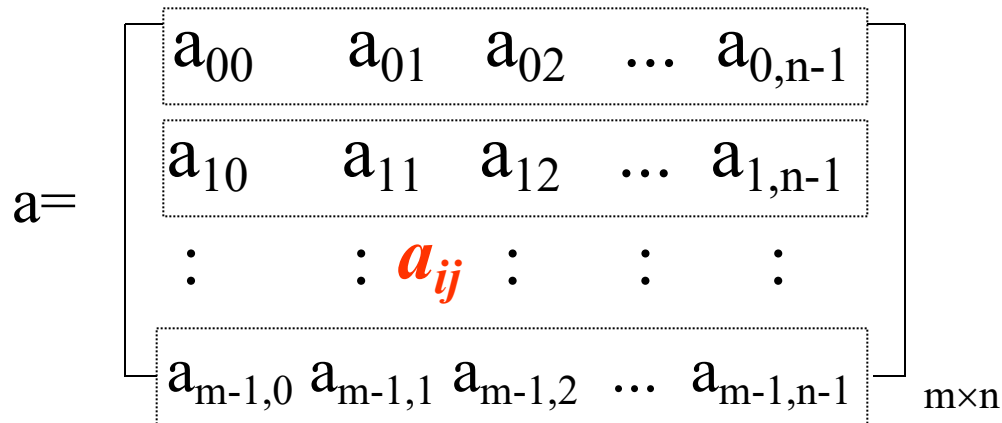
“行序为主序” 即 “低下标优先”

“列序为主序” 即 “高下标优先”

二维数组的顺序存储示意



二维数组的行序优先方式的顺序存储



二维数组 $a[m][n]$,各维元素个数为 m,n 。设数组开始存放位置
 $LOC(0,0) = b$ ，行序优先方式的顺序存储时， $a[i][j]$ 存放位置：

方法一 $LOC(i,j) = b + (i * n + j) * L$

通式（方法二）：

$$LOC[i,j] = LOC[0,0] + (i \times n + j) \times L = c_0 + c_1 \times i + c_2 \times j$$

$$\begin{cases} c_2 = L & // \text{一维数组单个数据元素大小} \\ c_1 = n \times c_2 = b_2 \times c_2 & // \text{二维数组的一行数据元素的全体大小} \\ c_0 = b = LOC[0,0] & // \text{起始地址} \end{cases}$$

练习

设有二维数组A[10,20]，其每个元素占两个字节，A[0][0]存储地址为100，若按行优先顺序存储，则元素A[6,6]的存储地址为352，按列优先顺序存储，元素A[6,6]的存储地址为232。

行优先：先存0-5行，共6行，每行20列；再存第6行，存了6列：

$$(6*20+6)*2+100=352$$

列优先：先存0-5列，共6列，每列10行；再存第6列，存了6行：

$$(6*10+6)*2+100=232$$

练习

设有一个二维数组 $A[m][n]$ 按行优先顺序存储，假设 $A[0][0]$ 存放位置在644， $A[2][2]$ 存放位置在676，每个元素占一个空间，问 $A[3][3]$ 存放在什么位置？

设数组元素 $A[i][j]$ 存放在起始地址为 $\text{Loc}(i, j)$ 的存储单元中

1、先算每行数据元素的个数 n ：

$$\because \text{Loc}(2, 2) = \text{Loc}(0, 0) + (2 * n + 2) * 1 = 644 + 2 * n + 2 = 676$$

$$\therefore n = (676 - 2 - 644) / 2 = 15$$

2、再算 $A[3][3]$ 存储位置

$$\therefore \text{Loc}(3, 3) = \text{Loc}(0, 0) + (3 * 15 + 3) * 1 = 644 + 45 + 3 = 692$$

n维数组

ElemType a[m₁][m₂] ... [m_n];

各维元素维度分别 $m_1, m_2, m_3, \dots, m_n$, 按维度先后次序进行顺序存储。
上述a数组按顺序存储 m_1 个n-1维的数组, 其中每个n-1维数组又是按顺序存储了 m_2 个n-2维的数组, 以此类推。

这样下标为 $i_1, i_2, i_3 \dots i_n$ 的n维数组元素的存储位置:

$$\begin{aligned} \text{方法一 } LOC(i_1, i_2, \dots, i_n) &= a + (i_1 * m_2 * m_3 * \dots * m_n \\ &\quad + i_2 * m_3 * m_4 * \dots * m_n \\ &\quad + \dots + i_{n-1} * m_n + i_n) * L \\ &= a + L * \sum_{j=1}^{n-1} i_j * \prod_{k=j+1}^n m_k \end{aligned}$$

通式（方法二）：

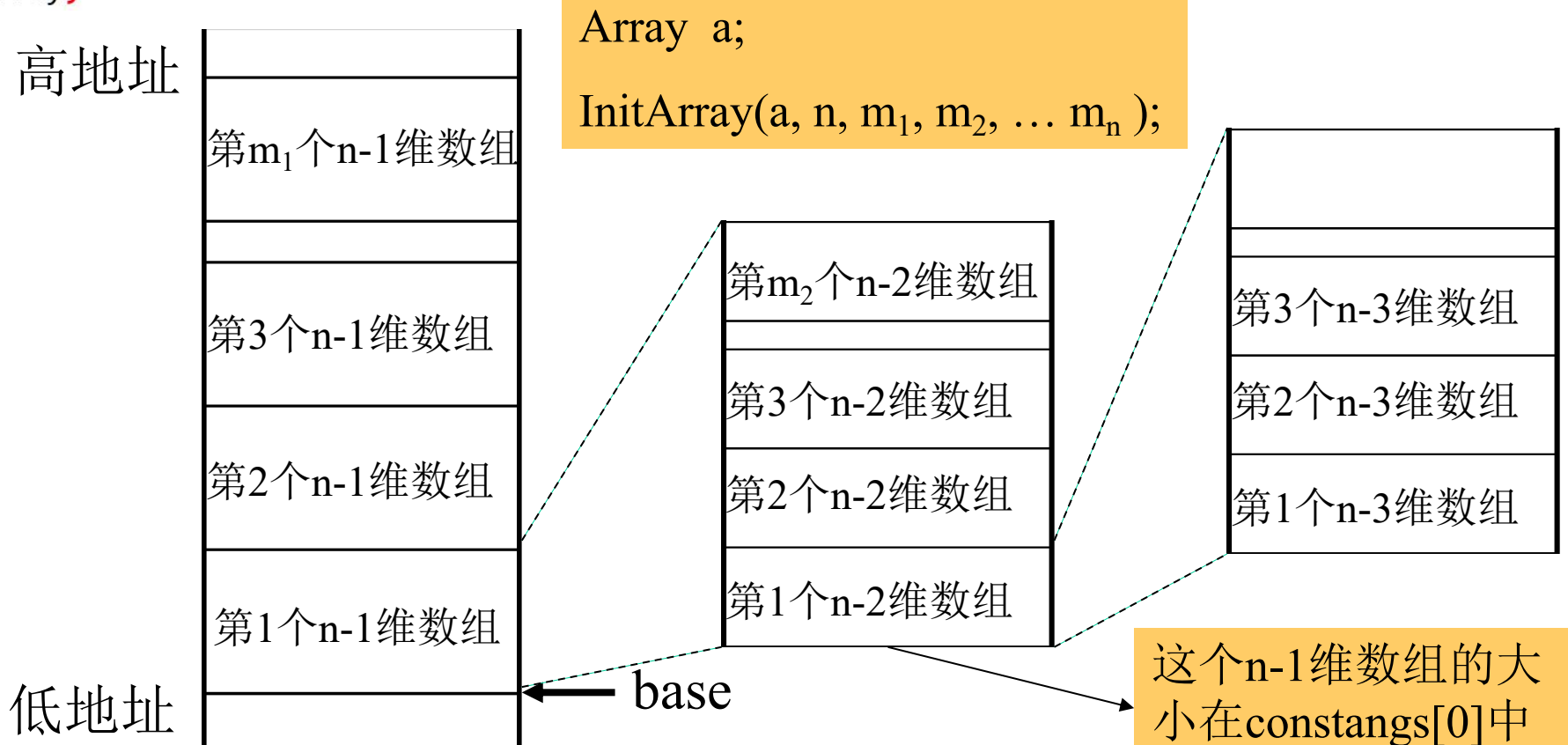
$$LOC[j_1, j_2, \dots, j_n] = c_0 + c_1 \times j_1 + c_2 \times j_2 + \dots + c_n \times j_n = c_0 + \sum_{i=1}^n c_i \times j_i$$

$$\text{其中: } \begin{cases} c_n = L; & // \text{ElemType大小} \\ c_{i-1} = c_i \times b_i \quad (1 < i \leq n) & // \text{n-i维数组整体大小} \\ c_0 = b = LOC[0, 0, \dots, 0] & // \text{基地址} \end{cases}$$

由LOC可知数组是一种随机存取结构:对任一元素存取时间相等。

```
#include <stdarg.h> //标准头文件, 提供宏va_start va_arg va_edn
//用于存取变长参数表

typedef struct{
    ElemType * base; //dim维数组元素的存储基地址, 在InitArray中分配, 在DestroyArrayz中释放
    int dim; //数组维度
    int * bounds; //数组维界基地址, 在InitArray中分配, 在DestroyArrayz中释放
    //Bounds, 初始化为整数数组, 存放m1, m2, ... mn 共dim个维度值
    int * constants; //数组映像函数常量基地址, 在InitArray中分配, 在DestroyArrayz中释放
    //constants, 初始化为整数数组, 存放LOC计算公式中的Ci值, 即各维数组整体大小
    //constants[dim-1]值为1, 即LOC中的Cn, ElemType类型数据元素大小为L
    //这里用整数1是利用了指针类型计算时, 指针值加n即相当于地址值+n*L的c语言机制
    //constants[i]值为constants[i+1]*bounds[i+1], 即LOC中的Ci
}Array;
```



```
Status InitArray(Array &A, int dim, ...) {
```

```
    // 若维数 dim 和各维长度合法,则构造相应的数组 A,并返回 OK
```

```
    if (dim < 1 || dim > MAX_ARRAY_DIM) return ERROR;
```

```
    A.dim = dim;
```

```
    A.bounds = (int *)malloc(dim * sizeof(int));
```

```
    if (!A.bounds) exit(OVERFLOW);
```

```
    // 若各维长度合法,则存入 A.bounds,并求出 A 的元素总数 elemtotal
```

```
    elemtotal = 1;
```

```
    va_start(ap, dim);          // ap 为 va_list 类型,是存放变长参数表信息的数组
```

```
    for (i = 0; i < dim; ++i) {
```

```
        A.bounds[i] = va_arg(ap, int);
```

```
        if (A.bounds[i] < 0) return UNDERFLOW;
```

```
        elemtotal *= A.bounds[i];
```

```
    }
```

```
    va_end(ap);
```

```
    A.base = (ElemType *)malloc(elemtotal * sizeof(ElemType));
```

```
    if (!A.base) exit(OVERFLOW);
```

```
    // 求映像函数的常数  $c_i$ ,并存入 A.constants[i-1],  $i = 1, \dots, \text{dim}$ 
```

```
    A.constants = (int *)malloc(dim * sizeof(int));
```

```
    if (!A.constants) exit(OVERFLOW);
```

```
    A.constants[dim-1] = 1;    // L=1,指针的增减以元素的大小为单位
```

```
    for (i = dim-2; i >= 0; --i)
```

```
        A.constants[i] = A.bounds[i+1] * A.constants[i+1];
```

```
    return OK;
```

```
}
```

```

Status Locate(Array A, va_list ap, int &off) {
    // 若 ap 指示的各下标值合法,则求出该元素在 A 中相对地址 off
    off = 0;
    for ( i = 0; i < A.dim; ++i ) {
        ind = va_arg(ap, int);
        if (ind < 0 || ind >= A.bounds[i]) return OVERFLOW;
        off += A.constants[i] * ind;
    }
    return OK;
}

```

```

Status Value(Array A, ElemType &e, ...) {
    // A 是 n 维数组, e 为元素变量, 随后是 n 个下标值。
    // 若各下标不超界, 则 e 赋值为所指定的 A 的元素值, 并返回 OK。
    va_start(ap, e);
    if ((result = Locate(A, ap, off)) <= 0) return result;
    e = *(A.base + off);
    return OK;
}

```

```

Status Assign(Array &A, ElemType e, ...) {
    // A 是 n 维数组, e 为元素变量, 随后是 n 个下标值。
    // 若下标不超界, 则将 e 的值赋给所指定的 A 的元素, 并返回 OK。
    va_start(ap, e);
    if ((result = Locate(A, ap, off)) <= 0) return result;
    *(A.base + off) = e;
    return OK;
}

```

Value和Assign调用Locate定位元素位置后进行取值和赋值。

注意Value和Assign中用va_start开始了可变长度参数处理, 将参数装入ap; 由函数参数传递在Locate中用va_arg处理ap的内容。

但是三个函数都没有使用va_end结束变长参数处理, va_end是将ap置为空指针, 可以不做。

思考题：4维数组ElemType A[3..7, 0..9, 1..6, 5..12], 设每数组元素占用8个存储单元，起始地址为1000, 求按低下标优先（类似行优先）顺序存储时，LOC[6,1,4,7]=?

解答：维数为 $m_1=5$, $m_2=10$, $m_3=6$, $m_4=8$ //上界减去下界加1

[方法一] LOC[6, 1, 4, 7]

$$\begin{aligned} &= 1000 + [10 \times 6 \times 8 \times (6-3) + 6 \times 8 \times (1-0) + 8 \times (4-1) + (7-5)] \times 8 \\ &= 13112 \end{aligned}$$

[方法二]相当于A[0..4, 0..9, 0..5, 0..7], 求LOC[3,1,3,2]

$$\begin{aligned} \text{LOC}[3,1,3,2] &= \text{LOC}[0,0,0,0] + c_1j_1 + c_2j_2 + c_3j_3 + c_4j_4 \\ &= 1000 + c_1j_1 + c_2j_2 + c_3j_3 + 8 \times 2 \\ &= 1000 + c_1j_1 + c_2j_2 + 8 \times 8 \times 3 + 8 \times 2 \\ &= 1000 + c_1j_1 + 8 \times 8 \times 6 \times 1 + 8 \times 8 \times 3 + 8 \times 2 \\ &= 1000 + 8 \times 8 \times 6 \times 10 \times 3 + 8 \times 8 \times 6 \times 1 + 8 \times 8 \times 3 + 8 \times 2 \\ &= 13112 \end{aligned}$$

5.3 矩阵的压缩存储

目的是节省空间。

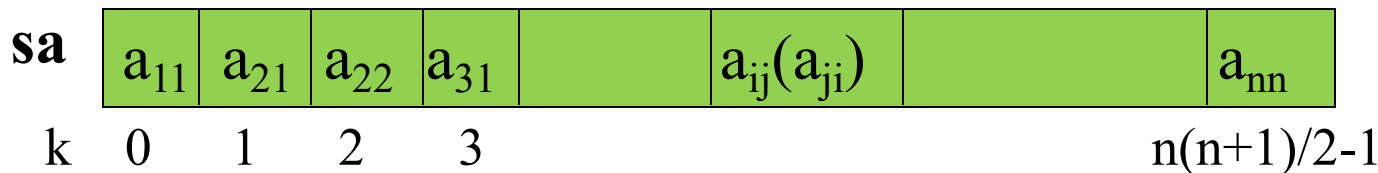
$$A = \begin{pmatrix} a_{11} & \dots & a_{1j} & a_{1N} \\ \dots & & & \dots \\ a_{i1} & & a_{ij} & a_{iN} \\ \dots & & & \dots \\ a_{M1} & \dots & a_{Mj} & a_{MN} \end{pmatrix}$$

5.3.1 对称矩阵

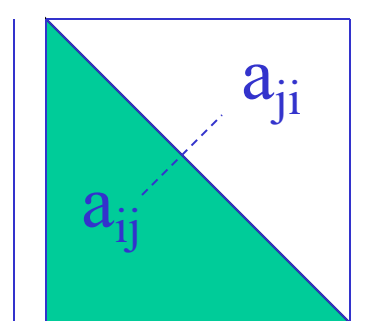
[特点] 在 $n \times n$ 的矩阵 a 中, 满足如下:

$$a_{ij} = a_{ji} \quad (1 \leq i, j \leq n)$$

[存储方法] 只存储下(或者上)三角(包括主对角线)的数据元素。共占用 $n(n+1)/2$ 个元素空间: $sa[0.. n(n+1)/2-1]$ 。

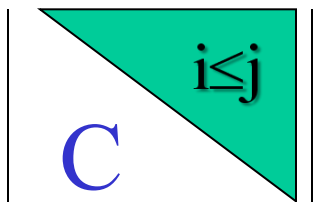


$$k = \begin{cases} i(i-1)/2 + j - 1 & \text{当 } i \geq j \\ j(j-1)/2 + i - 1 & \text{当 } i < j \end{cases}$$

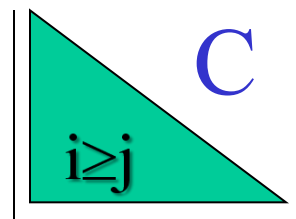


5.3.2 三角矩阵

[特点] 对角线以下(或者以上)的数据元素(不包括对角线)全部为常数 c 。



上三角矩阵



下三角矩阵

[存储方法] 重复元素 c 共享一个元素存储空间，共占用 $n(n+1)/2+1$ 个元素空间: $sa[0.. n(n+1)/2]$ 。 $sa[0]=C$



上三角矩阵

$$k = \begin{cases} (i-1) \times (2n-i+2)/2 + j - i + 1 & i \leq j \\ 0 & i > j \end{cases}$$

下三角矩阵

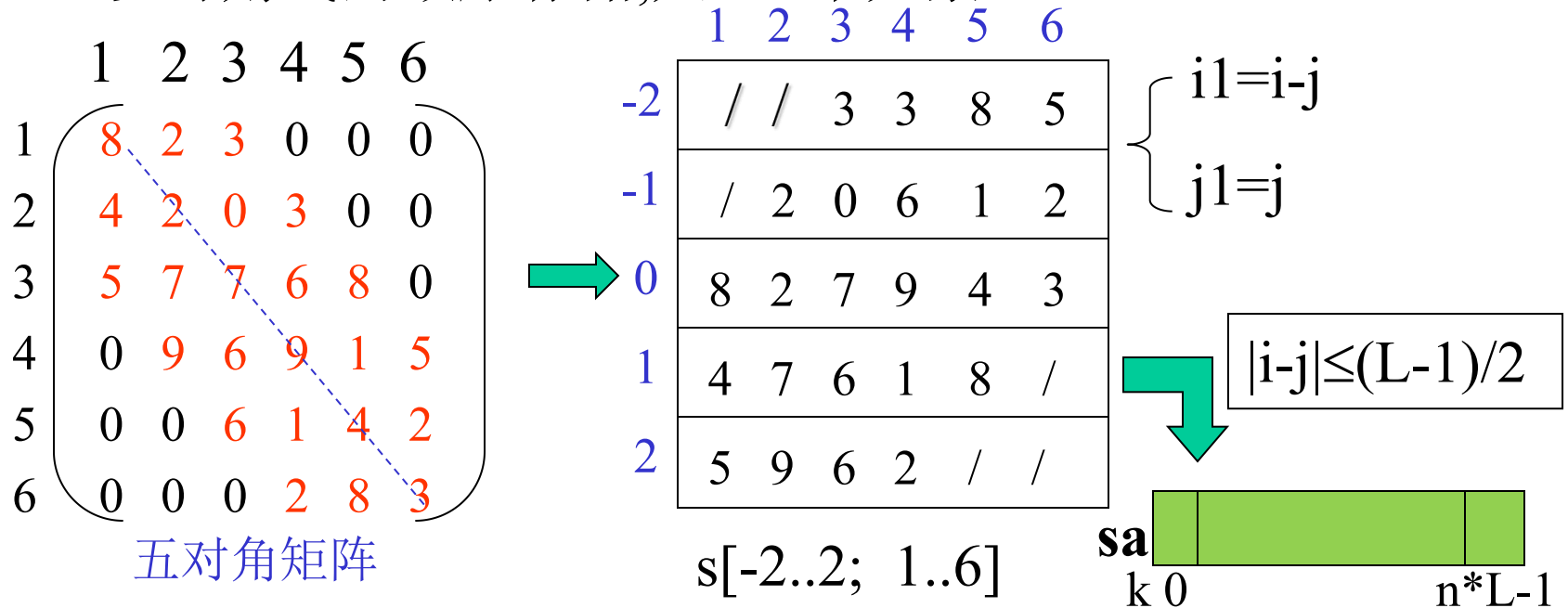
$$k = \begin{cases} i \times (i-1)/2 + j & i \geq j \\ 0 & i < j \end{cases}$$

5.3.3 带状矩阵（对角矩阵）

[特点] 在 $n \times n$ 的方阵中，非零元素集中在主对角线及其两侧共 L (奇数)条对角线的带状区域内 — L 对角矩阵。

[存储方法] 只存储带状区域内的元素。

1) 以对角线的顺序存储,共 $n * L$ 个元素。



2) 只存储带状区内的元素。除首行和末行, 按每行 L 个元素, 共 $(n-2)L + (L+1) = (n-1)L + 1$ 个元素。sa[0..(n-1)L]

存储地址计算: $k=(i-1)L+(j-i)$

$$1 \leq i, j \leq n \quad |i-j| \leq (L-1)/2$$

Diagram illustrating the construction of a 6x6 matrix A from a 6x6 matrix B .

Matrix B (left):

8	2	3	0	0	0
4	2	0	3	0	0
5	7	7	6	8	0
0	9	6	9	1	5
0	0	6	1	4	2
0	0	0	2	8	3

Matrix A (right):

8	2	3	/	4	2	0	3	5	7
7	6	8	9	6	9	1	5	6	1
4	2	/	2	8	3				

The dashed line indicates a mapping from the top-left element of B (8) to the bottom-right element of A (3).

5.3.4 随机稀疏矩阵

[特点] 大多数元素为零。 $\delta = t/(m \times n) \leq 0.05$

[常用存储方法] 记录每一非零元素(i, j, a_{ij})

——节省空间，但丧失随机存取功能

- 顺序存储：三元组表
- 链式存储：十字(正交)链表

15	0	0	22	0	-15
0	11	3	0	0	0
0	0	0	-6	0	0
0	0	0	0	0	0
91	0	0	0	0	0
0	0	28	0	0	0

5.3.4.1 三元组表

[类型定义]

```
#define MAXSIZE 1000
//设定非零元素最大值
typedef struct {
    int i, j;
    ElemType e;
}Triple;
typedef struct {
    Triple data[MAXSIZE+1];
    //三元组表, data[0]未用
    int mu, nu, tu;
    //行数、列数、非零元个数
}TSMatrix;
```

	i	j	e
1	1	1	15
2	1	4	22
3	1	6	-15
4	2	2	22
5	2	3	3
6	3	4	-6
7	5	1	91
8	6	3	28
m			

[应用例] 求转置矩阵

a.data

	i	j	e
1	1	1	15
2	1	4	22
3	1	6	-15
4	2	2	22
5	2	3	3
6	3	4	-6
7	5	1	91
8	6	3	28

转置



b.data

	i	j	e
1	1	1	15
2	1	5	91
3	2	2	22
4	3	2	3
5	3	6	28
6	4	1	22
7	4	3	-6
8	6	1	-15

算法1 { $O(n \times t)$ }

```
void TransMatrix(TSMatrix &b, TSMatrix a)
{
    b.mu=a.nu; b.nu=a.mu; b.tu=a.tu;
    if ( b.tu ) {
        q=1;    //指示b中存放数据的位置，初值为1
        for ( col=1; col<=a.nu; col++ )    //对a的每一列
            for ( p=1; p<=a.tu; p++ )    //对a的每个三元组检查
                if ( a.data[p].j==col ) {    //找列号为col的三元组
                    b.data[q].i =a.data[p].j;
                    b.data[q].j =a.data[p].i;
                    b.data[q].e =a.data[p].e;
                    q++;    //修正q值
                }
    }
} //TransMatrix
```

算法2 快速转置法 { $O(n+t)$ }

引入两个辅助向量：

- $\text{num}[1 : a.\text{nu}]$: a中每列的非零元素个数
- $\text{cpot}[1 : a.\text{nu}]$: a中每列的第一个非零元素在b中的位置

a中的列号

col num[col] cpot[col]

1	2	1
2	1	3
3	2	4
4	2	6
5	0	8
6	1	8

a	i	j	e		b	i	j	e
1	1	1	15	转置 →	1	1	1	15
2	1	4	22		2	1	5	91
3	1	6	-15		3	2	2	22
4	2	2	22		4	3	2	3
5	2	3	3		5	3	6	28
6	3	4	-6		6	4	1	22
7	5	1	91		7	4	3	-6
8	6	3	28		8	6	1	-15

$\text{cpot}[1]=1$

$\text{cpot}[\text{col}]=\text{cpot}[\text{col}-1]+\text{num}[\text{col}-1]$

$(2 \leq \text{col} \leq n)$

```

Statue FastTransMatrix(TSMatrix &b, TSMatrix a)
{
    b.mu=a.nu; b.nu=a.mu; b.tu=a.tu;
    if ( b.tu ) {
        for (col=1; col<= a.nu; ++col)    //num清零
            num[col]=0;
        for (t=1; t<=a.tu; ++t)           //对a.tu个非零元素按列号计数
            ++num[a.data[t].j];
        cpot[1]=1;                         //生成cpot
        for ( col=2; col<=a.nu; ++col )
            cpot[col]=cpot[col-1]+num[col-1];
        for ( p=1; p<=a.tu; ++p ) {       //对a.tu个非零元素转置
            col=a.data[p].j; q=cpot[col];
            b.data[q].i =a.data[p].j;
            b.data[q].j =a.data[p].i;
            b.data[q].e =a.data[p].e;
            ++cpot[col] ;
        }
    }
    return OK;
} //FastTransMatrix

```

5.3.4.2 行逻辑链接的表

便于随机存取任意一行的非零元素。

```
typedef struct {  
    Triple data[MAXSIZE+1];  
  
    int    rpos[MAXRC+1];  
  
    int    mu, nu, tu;  
}RLSMatrix;
```

RLSMatrix M;

M.rpos		M.data		
		i	j	e
1	1	1	1	15
2	4	1	4	22
3	6	1	6	-15
4	0	2	2	22
5	7	2	3	3
6	8	3	4	-6
		5	1	91
		6	3	28

这种方式可以便于某些运算，如：稀疏矩阵相乘等。

5.3.4.3 十字（正交）链表

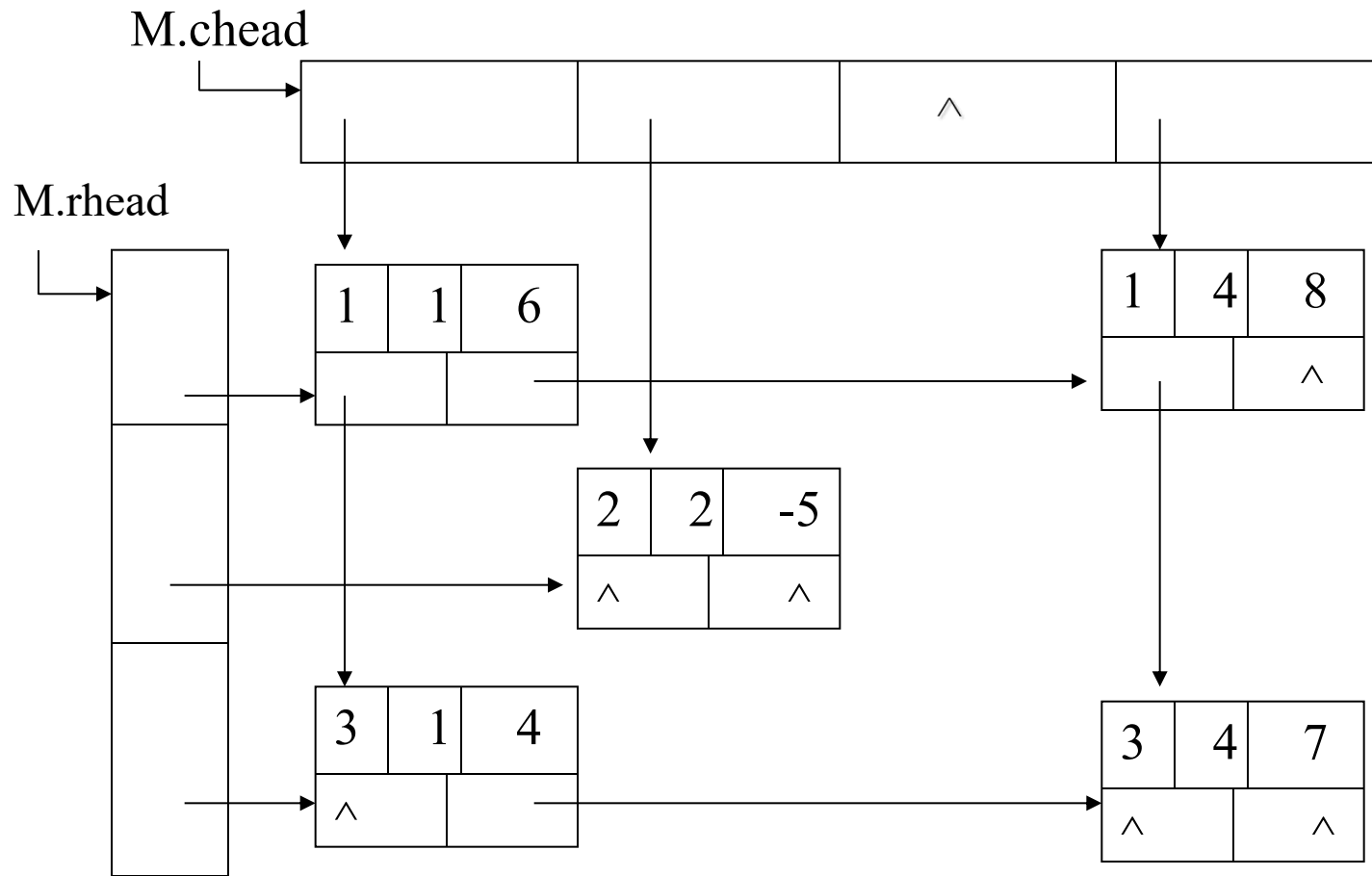
[特点] 在行、列两个方向上，将非零元素链接在一起。
克服三元组表在矩阵的非零元素位置或个数经常变动时的使用不便。

[类型定义]

```
typedef struct OLNode
{
    int      i, j;
    ElemType e;
    struct OLNode * right, * down;
} OLNode, * OLink;

typedef struct {
    OLink * rhead, * chead;
    int     mu, nu, tu;
} CrossList;
```

i	j	e
down		right



CrossList M;

6	0	0	8
0	-5	0	0
4	0	0	7

第五章 数组和广义表

广义表本质上是非线性结构，是扩展的线性表：表中的数据元素本身也是一个数据结构。

5.4 广义表的定义和表示方法

5.5 广义表的存储结构 *

5.6 广义表的递归算法 *

5.4 广义表（列表）的定义和表示方法

概念 广义表是由零个或多个原子或者子表组成的有限序列。可以记作： $LS=(d_1, d_2, \dots, d_n)$

原子：逻辑上不能再分解的元素。

子表：作为广义表中元素的广义表。

与线性表的关系

广义表中的元素全部为原子时即为线性表。线性表是广义表的特例，广义表是线性表的推广。

广义表的表示方法和相关术语

	表长	表深	表头	表尾
$A = ()$	0	1	-	-
$B = (a, A) = (a, ())$	2	2	a	(())
$C = ((a,b), c, d)$	3	2	(a,b)	(c,d)
$D = (a, D) = (a, (a, (a, \dots)))$	2	∞	a	(D)

表的长度 表中的元素（第一层）个数。

表的深度 表中元素的最深嵌套层数。

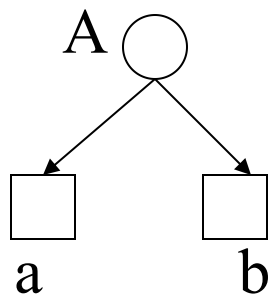
单原子的表，深度多少？

表头 表中的第一个元素。

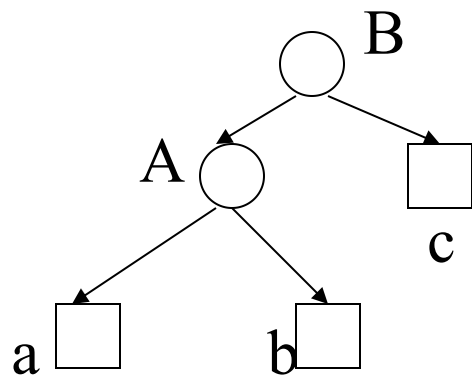
表尾 除第一个元素外，剩余元素构成的广义表。

非空广义表的表尾必定仍为广义表。

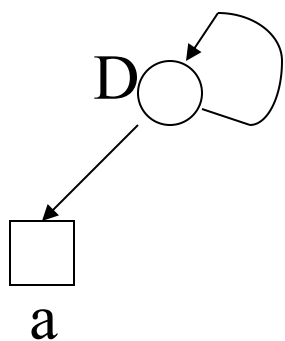
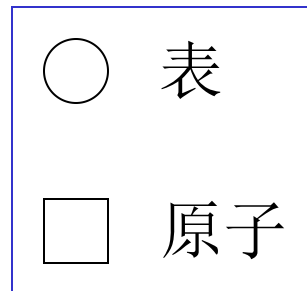
广义表的图形表达方法



$A=(a,b)$

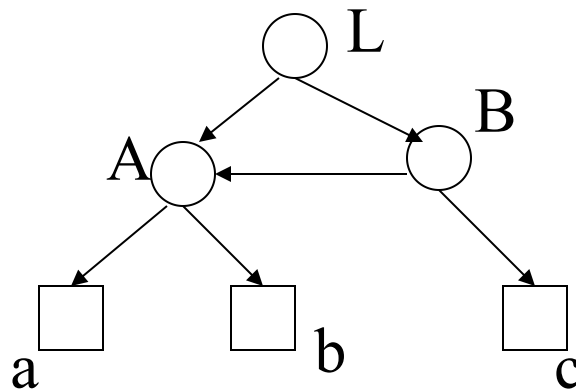


$B=(A,c)$



$D=(a,D)$

含有递归



$L=(A,B)$

含有共享

规定所有表都有名字时，广义表的表示方法

例 $A(a,b)$

$B(A(a,b), c)$

$L(A(a,b), B(A(a,b),c))$

$D(a, D(a, D(...)))$

广义表结构的性质

- 纯表：与树
 - 再入表：允
 - 递归表：允
- 递归表 $\supset$ 再入表

$$\begin{aligned} P(x, y, z) &= x^{10}y^3z^2 + 2x^6y^3z^2 + 3x^5y^2z + 2yz + 15 \\ &= (x^{10}y^3 + 2x^6y^3)z^2 + (3x^5y^2 + 2y)z + 15 \end{aligned}$$

$$P = z((A, 2), (B, 1), (15, 0))$$

$$A = y((C, 3))$$

$$C = x((1, 10), (2, 6))$$

$$B = y((D, 2), (2, 1))$$

$$D = x((3, 5))$$

广义表的应用

如：程序的语句结构；m元多项式的表示

基本操作

- 结构的创建和销毁

InitGList(&L); DestroyGList(&L);
CreateGList(&L, S); CopyGList(&T, L);

- 状态函数

GListLength(L); GListDepth(L);
GListEmpty(L); GetHead(L); GetTail(L);

- 插入和删除操作

InsertFirst_GL(&L, e);
DeleteFirst_GL(&L, &e);

- 遍历

Traverse_GL(L, Visit());

抽象数据类型定义

ADT GList {

数据对象： $D = \{e_i \mid i=1,2,\dots,n; n \geq 0; e_i \in \text{AtomSet} \text{ 或 } e_i \in \text{GList}, \text{AtomSet 为某个数据对象}\}$

数据关系: $R = \{ \langle e_{i-1}, e_i \rangle \mid e_{i-1}, e_i \in D, 2 \leq i \leq n \}$

基本操作:

```
InitGList(&L);
```

操作结果：创建空的广义表L。

```
CreateGLList(&L, S);
```

初始条件: S是广义表的书写形式串。

操作结果：由S创建广义表L。

• • • • •

} ADT GList

5.5 广义表的存储结构

5.5.1 方法 1 — 头尾链表形式

[类型定义]

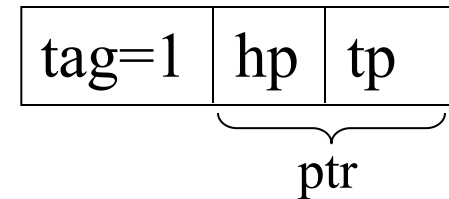
```
typedef enum {ATOM, LIST} ElemTag;  
// ATOM==0: 原子; LIST==1: 子表  
typedef struct GLNode{  
    ElemTag tag;  
    union {  
        AtomType atom;  
        struct {  
            struct GLNode *hp, *tp;  
        } ptr;  
    }  
} * GList1;
```

$GL = (a_1, a_2, \dots, a_n)$

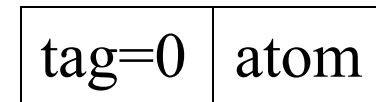
$\text{head}(GL) = a_1$

$\text{Tail}(GL) = (a_2, \dots, a_n)$

表结点



原子结点

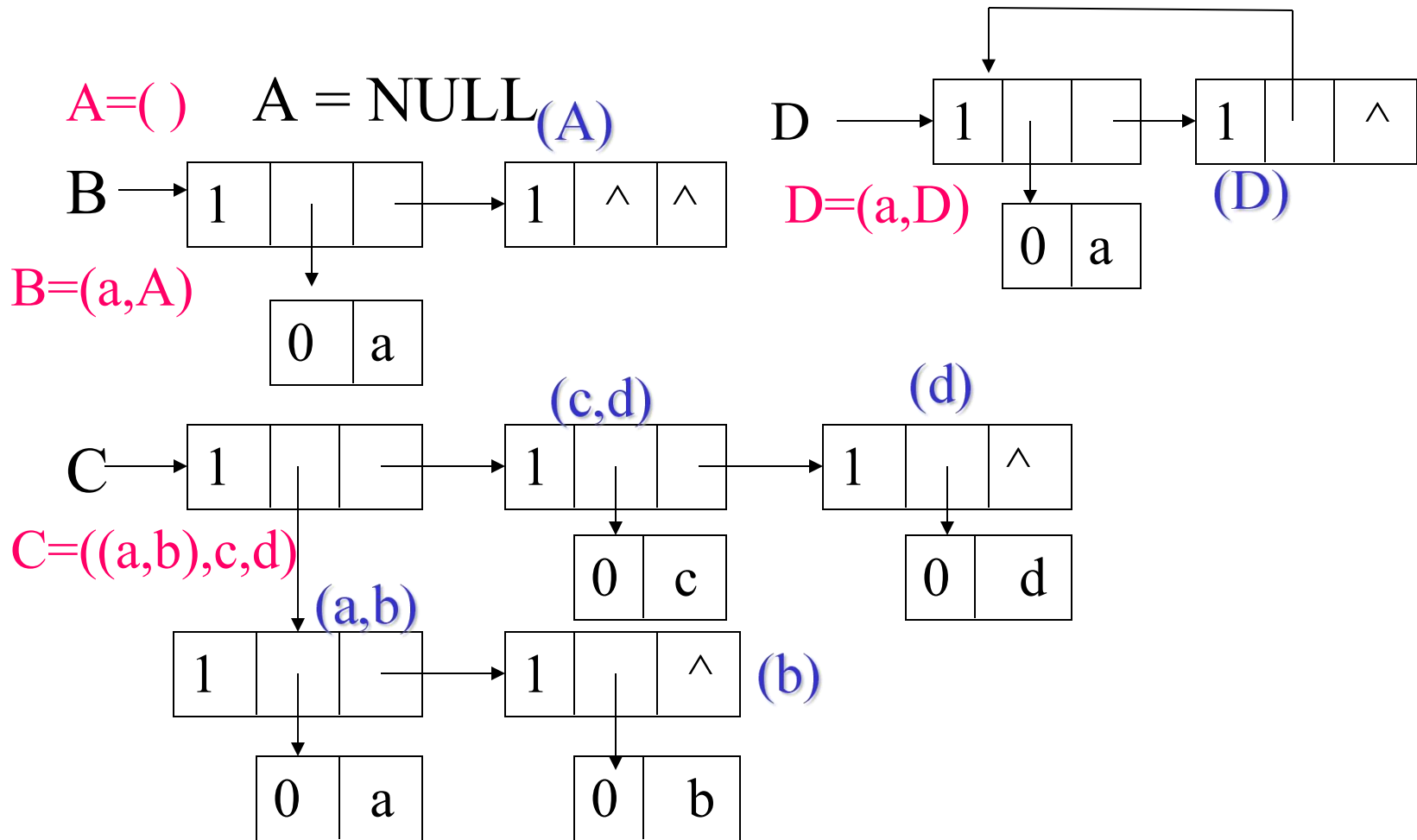


hp: 指示表头的指针

tp: 指示表尾的指针

[示例]

GList1 A, B, C, D



5.5.2 方法2—扩展的线性链表形式

[类型定义]

$(a_1, a_2, \dots, a_n)$

```
typedef enum {ATOM, LIST} ElemTag;
// ATOM==0: 原子; LIST==1: 子表
typedef struct GLNode{
    ElemTag tag;
    union {
        AtomType atom;
        struct {
            struct GLNode *hp;
        }
    }
    struct GLNode *tp;
} * GList2;
```

表结点

tag=1	hp	tp
-------	----	----

原子结点

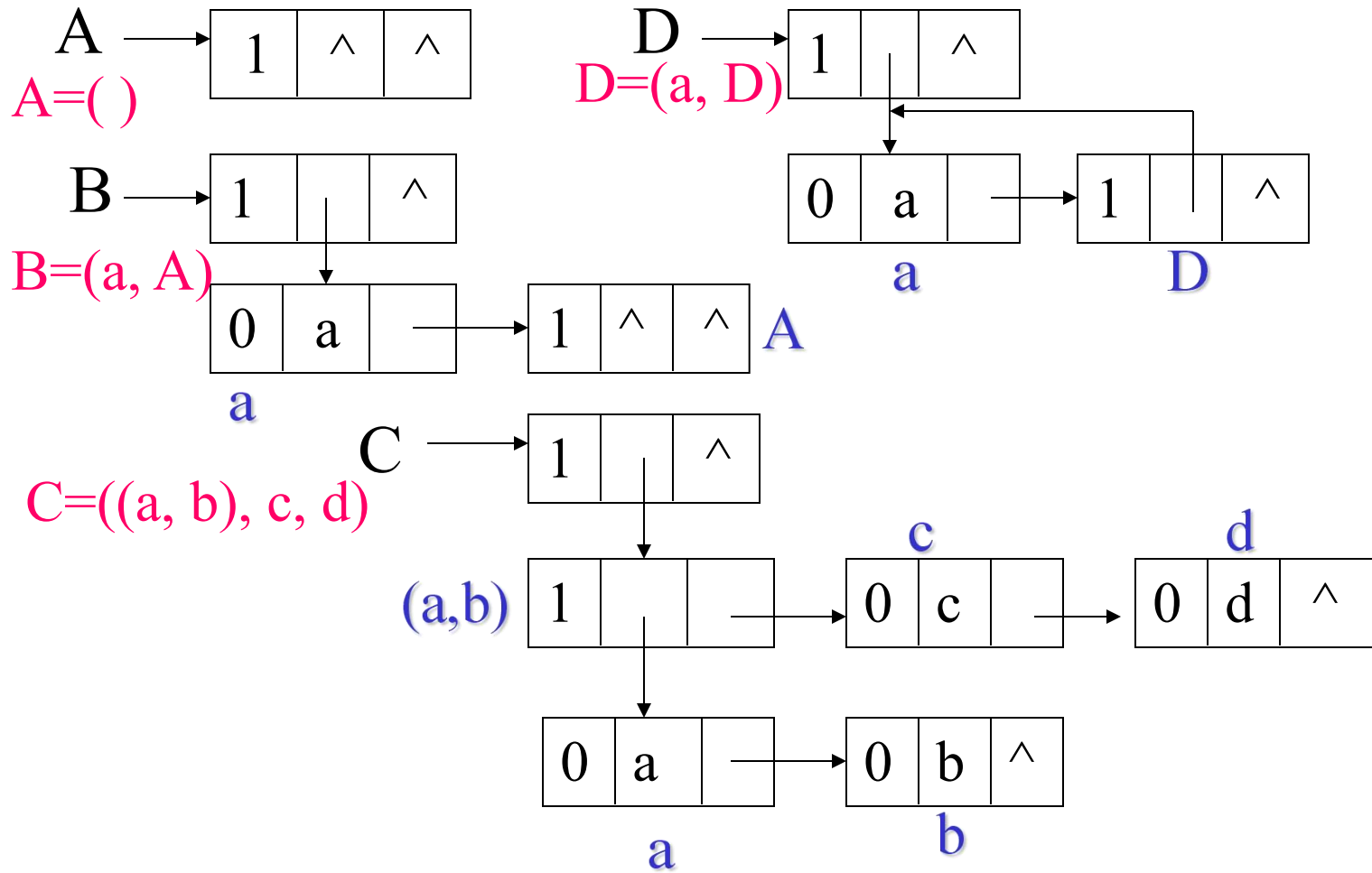
tag=0	atom	tp
-------	------	----

hp: 指向表头的指针

tp: 指向同一层的下一个结点; 构成一个单链表, 把本层的原子或主表都链起来了。

[示例]

GList2 A, B, C, D



5.6 广义表的递归算法

广义表的特点：定义是递归的。

示例约定：非递归表且无共享子表。

[示例1] 计算广义表的深度

方法一 分析表中各元素（子表） $LS = (a_1, a_2, \dots, a_n)$

– 求深度的递归函数

基本项： $DEPTH(LS) = 1$ 当LS为空表时

$DEPTH(LS) = 0$ 当LS为原子时

归纳项： $DEPTH(LS) = 1 + \max_{1 \leq i \leq n} \{DEPTH(a_i)\} \quad n \geq 1$

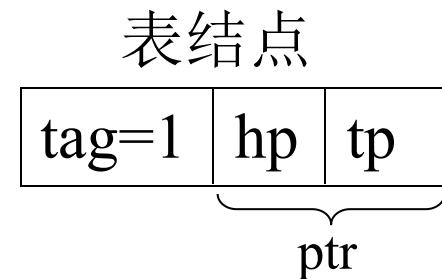
– 算法描述

```

int GListDepth(GList1 L)
{
    if (!L) return 1; //空表
    if (L->tag==ATOM) return 0; //单原子
    for (max=0, pp=L; pp; pp=pp->ptr.tp) {
        dep=GListDepth (pp->ptr.hp);
        if (dep>max) max=dep;
    }
    return max+1;
} //GListDepth

```

$LS = (a_1, a_2, \dots, a_n)$
 表头: a_1
 表尾: $(a_2, \dots, a_n)$



方法二 分析表头和表尾

– 求深度的递归函数

$$\text{depth}(\text{LS}) = \begin{cases} 1 & \text{当LS为空表} \\ 0 & \text{当LS为单原子} \\ \max\{\text{depth}(\text{head}(\text{LS}))+1, \text{depth}(\text{tail}(\text{LS}))\} & \text{其它情况} \end{cases}$$

– 算法描述

```
int GListDepth(GList1 L)
{
    if (!L) return 1; //空表
    if (L->tag==ATOM) return 0; //单原子
    dep1=GListDepth (L->ptr.hp)+1;
    dep2=GListDepth (L->ptr.tp);
    if (dep1>dep2) return dep1;
    else return dep2;
} //GListDepth
```

[示例2] 复制广义表

– 复制操作的递归定义

基本项: $\text{InitGList}(\text{NEWLS})$

当LS为空表时;

复制单原子结点

当LS为原子时;

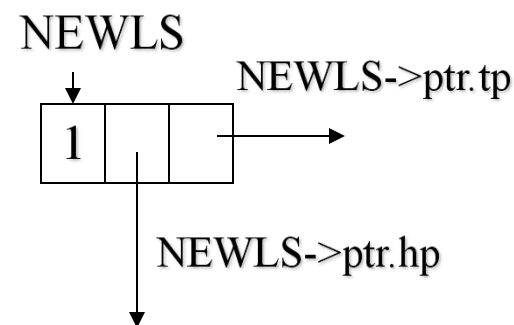
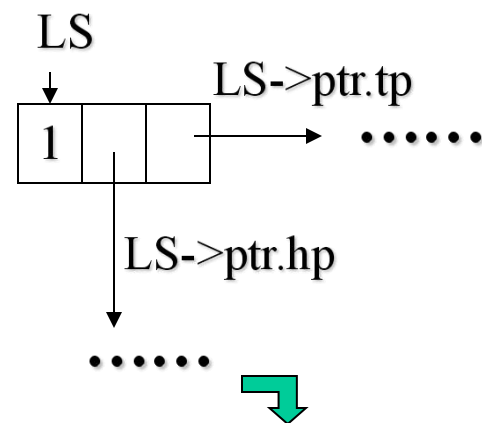
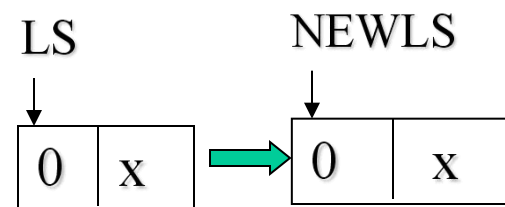
归纳项: 建表结点;

复制表头;

复制表尾;

– 算法描述CopyGLists

$\text{LS} = \text{NULL} \rightarrow \text{NEWLS} = \text{NULL}$



```

Status CopyGList(GList1 &T, GList1 L)
{
    if (!L) T=NULL; //复制空表
    else {
        T=(GList1)malloc(sizeof(GLNode));
        if (!T) exit(OVERFLOW);
        T->tag=L->tag;
        if (L->tag==ATOM) T->atom=L->atom; //复制单原子
        else {
            CopyGList(T->ptr.hp, L->ptr.hp);
            CopyGList(T->ptr.tp, L->ptr.tp);
        }
    }
    return OK;
} //CopyGList

```

本章学习要点

- 掌握数组类型的特点以及在高级编程语言中的两种存储表示和实现方法，并熟练掌握低下标优先时指定下标的元素在存储结构中的地址计算方法。
- 掌握矩阵压缩存储的常用方法。
- 掌握广义表的结构特点和表长、表深、表头、表尾的定义。
- 了解广义表的存储表示方法，学会对非空广义表进行分解的两种分析方法：即可将一个非空广义表分解为表头和表尾两部分或者分解为 n 个子表。巩固递归算法的设计思想。*